ACADEMIC YEAR: 2025-26

Course: Skill base Lab Course: Cloud Computing

Course code: CSL605

Year/Sem: TE/VI

Experiment No.: 09
Aim: To study and Implement Containerization using Docker
Name: Shravani Sandeep Raut
Roll Number: 40
Date of Performance: 16/03/2026
Date of Submission: 18/03/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	2
Understanding	4-5	2-3	2
Journal work and timely submission.	8-10	5-8	4

Checked by

Name of Faculty : Dr. Tatwadarshi Nagarhalli

Signature :

Date :

Experiment No. 9

Aim: To study and Implement Containerization using Docker

Objective: To know the basic differences between Virtual machine and Container. It involves demonstration of creating, finding, building, installing, and running Linux/Windows application containers inside a local machine or cloud platform.

Theory:

- open platform for developing, shipping and running applications
- enables you to separate your applications from your infrastructure so you can deliver software quickly
- you can manage your infrastructure in the same ways you manage your applications
- Docker provides the ability to package and run an application in a loosely isolated environment called a container.
- Containers are lightweight and contain everything needed to run the application, so you do not need to rely on what is currently installed on the host.
- Develop your application and its supporting components using containers.
- The container becomes the unit for distributing and testing your application.
- When you're ready, deploy your application into your production environment, as a container or an orchestrated service. This works the same whether your production environment is a local data center, a cloud provider, or a hybrid of the two.

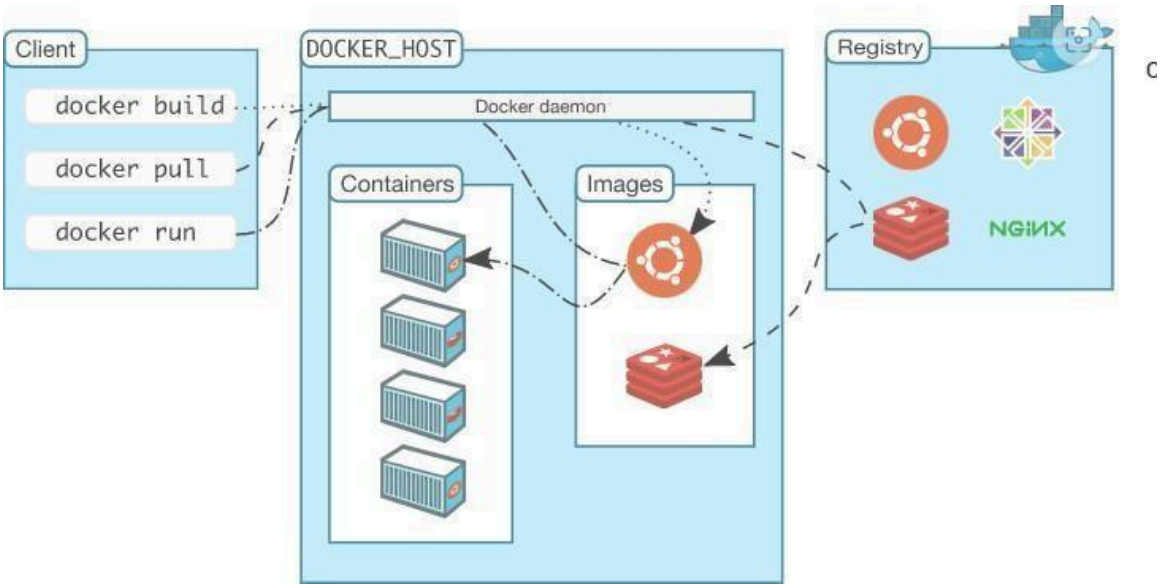
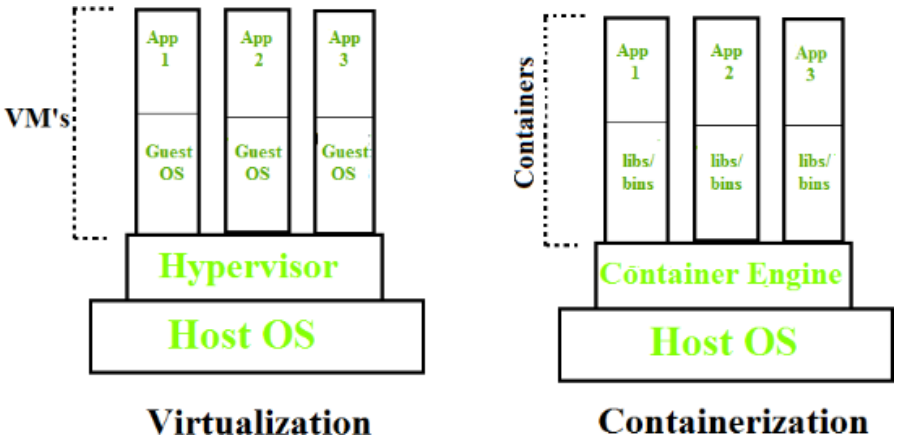


Figure 1: Docker Architectur

Containerization:

- Containerization is OS-based virtualization that creates multiple virtual units in the user space, known as Containers.
- Containers share the same host kernel but are isolated from each other through private namespaces and resource control mechanisms at the OS level.
- Container-based Virtualization provides a different level of abstraction in terms of virtualization and isolation when compared with hypervisors.
- Hypervisors use a lot of hardware which results in overhead in terms of virtualizing hardware and virtual device drivers.
- containers implement isolation of processes at the operating system level, thus avoiding such overhead.
- Containerization has better resource utilization compared to VMs and a short boot-up process. It is the next evolution in virtualization.
- Containers can run virtually anywhere, greatly easy development and deployment: on Linux, Windows, and Mac operating systems; on virtual machines or bare metal, on a developer's machine or in data centers on-premises; and of course, in the public cloud.
- Containers virtualize CPU, memory, storage, and network resources at the OS level, providing developers with a sandboxed view of the OS logically isolated from other applications.
- Docker is the most popular open-source container format available and is supported on Google Cloud Platform and by Google Kubernetes Engine.



Steps:

1. Open **docker.com**

2. Scroll down and click on **"Get started for free"**

3. Click on **Docker Desktop** and download it

4. After downloading, open **Docker Desktop Installer** and start installation

5. After installation, **restart your device**

6. Accept the **terms and conditions**

7. Click on the link: **https://aka.ms/wsl2kernel** *(do not close Docker window)*

8. Download the **WSL2 Linux kernel update package (x64)**

9. After download, run the **.msi package**

10. Click **Next → Finish**

11. Open **PowerShell as Administrator**

12. Run command:

wsl --set-default-version 2

13. Restart system again

14. Open Docker and click **Start**

Commands

1. To check Docker version
docker --version

2. To install Ubuntu image
docker pull ubuntu

3. To check downloaded images
docker images

4. To run Ubuntu container
docker run -it ubuntu /bin/bash

5. Open another Command Prompt

6. To check running containers
docker ps

7. To list all containers
docker container ls -a

8. To stop container
docker container stop <container_id>

9. To remove container
docker container rm <container_id>

10. To verify running containers
docker ps

11. To list all Docker commands
docker

12. To remove Docker image
docker image rm <image_id>

13. To run Ubuntu again
docker run -it ubuntu /bin/bash

Output/Observation:

```
C:\Users\student>docker --version
Docker version 25.0.3, build 4debf41

C:\Users\student>
```

```
PS C:\WINDOWS\system32> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
PS C:\WINDOWS\system32>
```

```
:\Users\student>docker pull ubuntu
Using default tag: latest
atest: Pulling from library/ubuntu
01d7766a2e4a: Pull complete
Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

```
C:\Users\student>docker images
REPOSITORY                TAG       IMAGE ID       CREATED        SIZE
ubuntu                    latest    bbdabce66f1b   4 weeks ago    78.1MB
myfirstpythonapp          latest    7e0859afd699   23 months ago  1.02GB
docker/welcome-to-docker  latest    c1f619b6477e   2 years ago    18.6MB

C:\Users\student>
```

```
C:\Users\student>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES

C:\Users\student>docker container ls -a
"docker container ls" accepts no arguments.
See 'docker container ls --help'.

Usage:  docker container ls [OPTIONS]

List containers
```

```
C:\Users\student>docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS         PORTS     NAMES
d0abc14a4181   ubuntu    "/bin/bash"   2 minutes ago   Up 2 minutes             fervent_cannon

C:\Users\student>docker container ls -a
CONTAINER ID   IMAGE                          COMMAND                  CREATED             STATUS                      PORTS
NAMES
2a1e079c9db1   ubuntu                         "/bin/bash"              About a minute ago  Exited (127) 20 seconds ago
focused_dijkstra
d0abc14a4181   ubuntu                         "/bin/bash"              2 minutes ago       Up 2 minutes
fervent_cannon
ee804a37650f   docker/welcome-to-docker:latest  "/docker-entrypoint.…"  11 months ago       Exited (255) 4 months ago   0.0.0.0:8088->80/tcp
welcome-to-docker

C:\Users\student>docker container stop 2a1e079c9db1
2a1e079c9db1

C:\Users\student>docker container rm 2a1e079c9db1
2a1e079c9db1
```

```
C:\Users\student>docker container stop 2a1e079c9db1
2a1e079c9db1

C:\Users\student>docker container rm 2a1e079c9db1
2a1e079c9db1

C:\Users\student>docker images
REPOSITORY                TAG       IMAGE ID       CREATED        SIZE
ubuntu                    latest    bbdabce66f1b   4 weeks ago    78.1MB
myfirstpythonapp          latest    7e0859afd699   23 months ago  1.02GB
docker/welcome-to-docker  latest    c1f619b6477e   2 years ago    18.6MB

C:\Users\student>docker image rm ff0fea8310f3
Error response from daemon: No such image: ff0fea8310f3:latest

C:\Users\student>docker run -it ubuntu /bin/bash
root@33b451f5fa6b:/# exit
exit
```

Conclusion:

The implementation of containerization using Docker was successfully carried out by installing Docker Desktop, pulling the required image, and creating and managing containers. Docker enables lightweight virtualization by packaging applications along with their dependencies into containers, ensuring consistency across different environments.